MiniLab 6: Neural Network Regression

Why Classic ML Still Wins on This Tabular Housing Dataset

Rizaldy
rutomo@andrew.cmu.edu
Machine Learning Foundations with Python

Spring 2026

1 Introduction

This report summarizes the main modeling result from MiniLab 6: even after tuning, the neural network improved substantially but still did not beat the Random Forest on this housing-price prediction task. That does not mean neural networks are weak in general. It means this dataset has the profile where classic tree-based methods usually remain very strong: structured tabular features, moderate sample size, mixed numeric and categorical inputs, and a target shaped by many threshold-like interactions.

2 Experimental Setup

The notebook uses the same train/test split for both models, with 2,829 rows total, 1,980 training rows, and 849 test rows. MAPE is the primary metric, with RMSE, MAE, and R-squared as supporting metrics.

Table 1: Baseline model comparison on the shared test set.

Model	MAPE	R-squared
Random Forest	10.27%	0.879
Default Neural Network (100,)	19.80%	0.640

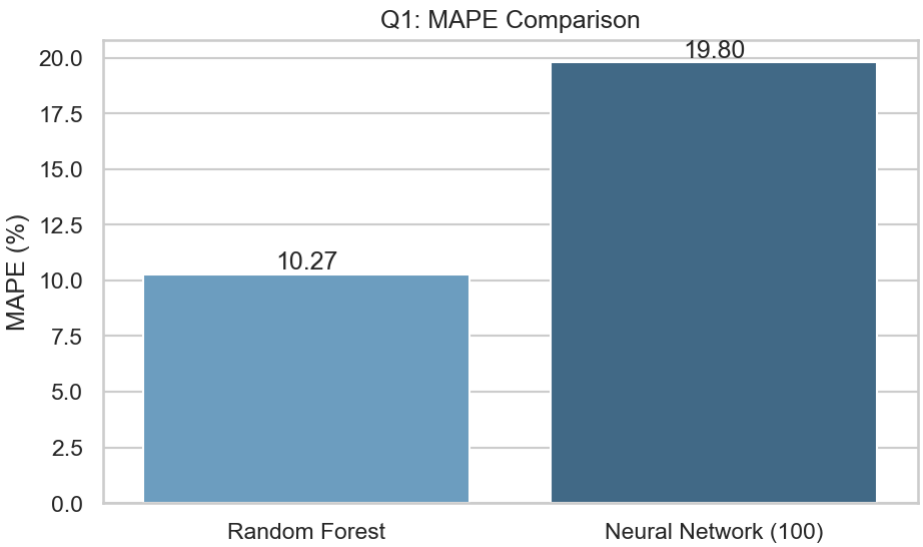

Figure 1: Random Forest opens with a large MAPE advantage over the default neural network.

The gap is large enough that this is not just noise. The Random Forest is already strong before tuning, while the default neural network is not yet competitive.

3 Why Classic ML Still Wins Here

3.1 The data is tabular, not representation-heavy

After preprocessing, the model sees 272 one-hot or numeric features. This is very different from images or raw text, where neural networks often dominate because they can learn deep representations directly from unstructured inputs.

Here, much of the representation work is already done by the columns: lot size, living area, room counts, neighborhood indicators, remodeling features, and similar variables. A Random Forest can exploit those feature splits immediately. It does not need to discover a hierarchy of latent features first.

3.2 The sample size is still small for a dense neural network

There are enough rows to train a useful regression model, but not so many that a dense neural network with tens of thousands of weights has an easy optimization problem. The best three-layer network in the notebook uses 33,651 parameters. That is a lot of flexibility relative to 1,980 training rows.

The forest is a better match to this scale. It can fit nonlinear patterns without asking gradient descent to tune thousands of continuous weights from a comparatively modest dataset.

3.3 Housing prices contain threshold-like interactions

Housing values often move through local rules rather than one smooth function:

- value jumps after certain living-area ranges,
- garage or basement effects depend on neighborhood,
- remodeling matters unevenly across homes,
- categorical and numeric variables interact in irregular ways.

That type of structure is naturally handled by trees. A forest partitions the feature space into many local rules quickly. A neural network can approximate the same behavior, but only with the right scaling, architecture, learning rate, and stopping behavior.

Figure 2: Living area matters, but the wide spread shows price depends on many irregular local interactions beyond a single smooth trend.

3.4 Random Forest is more robust to messy tabular inputs

The forest baseline does not need feature scaling and is generally tolerant of mixed input distributions. The neural network is much more sensitive. In the notebook, the NN only became respectable after deliberate tuning. That sensitivity is part of the finding: on this kind of data, a model that works well with less tuning has a practical advantage.

4 What the Neural Network Did Learn

The neural network was not a failure. It improved meaningfully once the architecture and optimizer were tuned.

Table 2: Neural-network architecture sweep.

Architecture	**MAPE**
Small $(32,)$	24.73%
Medium $(100,)$	19.80%
Large $(200,)$	18.62%
Two layers $(100, 50)$	15.55%
Three layers $(100, 50, 25)$	14.45%

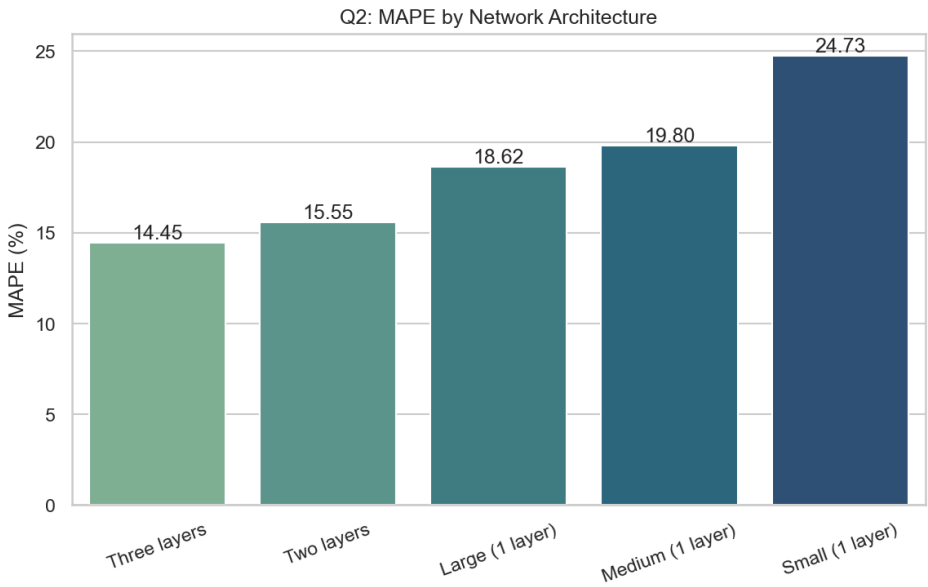

Figure 3: Deeper architectures steadily improve neural-network accuracy, but they do not overtake the Random Forest baseline.

The prompt also asks for the explicit parameter counts of the Small and Three Layers networks. Using 272 input features:

- **Small** $(32,)$**:** $(272 \times 32) + 32 + (32 \times 1) + 1 = 8{,}769$ parameters
- **Three layers** $(100, 50, 25)$**:** $(272 \times 100) + 100 + (100 \times 50) + 50 + (50 \times 25) + 25 + (25 \times 1) + 1 = 33{,}651$ parameters

That pattern matters. The deeper models clearly help, which means the network is learning genuine higher-order interactions. But the gain is gradual, not transformative. Better architecture narrows the gap; it does not reverse the ranking.

Epoch behavior is also part of the required analysis:

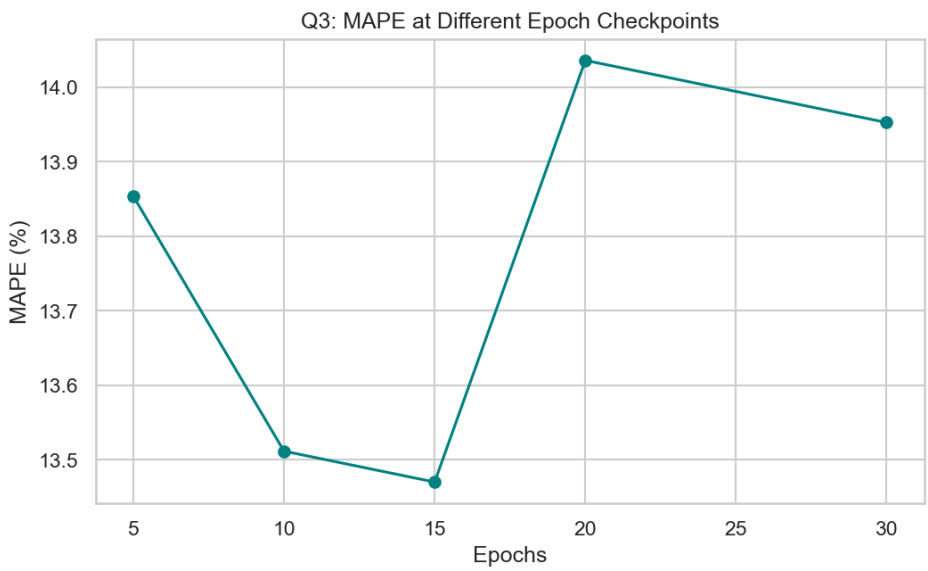

Figure 4: Epoch sweep for the best architecture. Error falls early and then flattens, suggesting convergence happens fairly quickly in this setup.

The epoch sweep shows that error drops quickly at first and then flattens. The best values in this short sweep appear around 10 to 15 epochs, while 20 and 30 epochs do not improve further. That suggests convergence happens fairly early on this setup, and simply training longer is not the main lever for better performance.

Learning rate mattered even more:

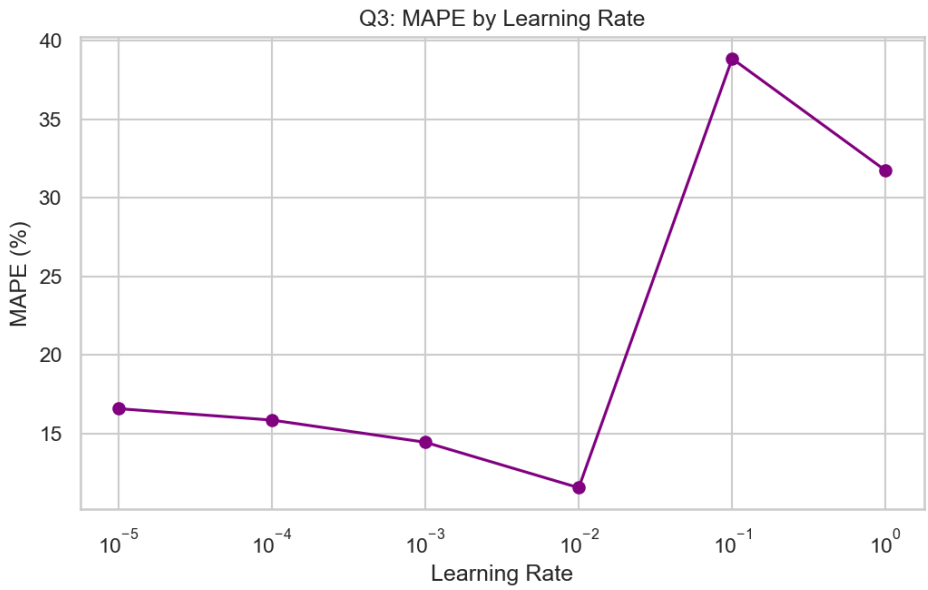

Figure 5: Learning rate tuning improves the best NN result to 11.56% MAPE at 0.01.

The best tuned neural network reached **11.56% MAPE** at learning rate `0.01`. That is a strong recovery from the default 19.80%, and it shows the network had enough capacity to become competitive. Still, even after tuning, the Random Forest remained slightly better at **10.27%**.

This is the most important reflective point in the notebook: tuning can rescue a neural network, but rescue is not the same as winning.

5 Reflection

The final result is a good reminder not to overvalue newer models just because they are more flexible.

For this problem:

1. the feature engineering already did much of the representation work,
2. the dataset size favors a strong tabular baseline,
3. the target depends on irregular, local interactions,
4. and the simpler model reaches the better test error with less tuning burden.

That is exactly the situation where classic ML should still be taken seriously. Random Forest wins here not because it is more modern or more powerful in the abstract, but because it is better aligned with the structure of the data.

The neural network still adds value as a learning exercise. It shows how architecture depth and optimizer settings affect regression performance, and it demonstrates that the gap can be narrowed substantially. But if the goal were purely to ship the best tabular predictor from this notebook, the Random Forest would still be the safer choice.